Green Software Engineering in Practice: Quantifying the Impact of Carbon Reduction Strategies in Enterprise Software

Rutger Kool
Vrije Universiteit Amsterdam
The Netherlands
r.g.kool@student.vu.nl

Michiel Overeem
AFAS Software
The Netherlands
michiel.overeem@afas.nl

Ivano Malavolta
Vrije Universiteit Amsterdam
The Netherlands
i.malavolta@vu.nl

Abstract

Context. Measuring and optimizing software's carbon footprint is methodologically challenging. While the Software Carbon Intensity (SCI) specification standardizes carbon measurement, most research emphasizes infrastructure- over application-level optimizations.
Goal. We aim to evaluate carbon emission reduction strategies for an industrial software application by applying the SCI specification.
Method. This study targets AFAS SB, an enterprise application developed in the Netherlands. Firstly, baseline carbon emissions are assessed via the SCI specification from Microsoft Azure metrics collected in production. Then, in-the-lab experiments are conducted for quantifying how 18 application-level configurations impact the estimated CO_2 emissions of AFAS SB across 79 scenarios.
Results. We provide a concrete structured approach for applying the SCI specification to evaluate carbon emissions in enterprise applications. Garbage collection optimization reduced CO_2 emissions by 29.5–30.6%, while other configurations had minimal effect. Combining top parameters achieved a 25.7% reduction without compromising functionality. Correlation analysis linked disk I/O reads to higher emissions.
Conclusions. Our findings show that developers can reduce CO_2 emissions through application-level configuration changes, with SCI enabling standardized measurement of optimization effectiveness across scenarios.

ACM Reference Format:
Rutger Kool, Michiel Overeem, and Ivano Malavolta. 2026. Green Software Engineering in Practice: Quantifying the Impact of Carbon Reduction Strategies in Enterprise Software. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-SEIP '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3786583.3786850

1 Introduction

Organizations face growing pressure to address their environmental impact through reduced energy consumption and carbon emissions, driven by concerns over global warming [28]. Assessing the energy consumption of cloud-native systems is complicated by the abstraction introduced by virtualization and orchestration layers [4]. Estimation techniques have been developed that map container-level metrics [35], while telemetry-based methods support runtime observation of distributed behavior and provide additional insights into energy-related activity [53]. To convert energy measurements into carbon emissions, the Software Carbon Intensity (SCI) specification uses energy consumption data from software systems. SCI is a standard (ISO/IEC 21031:2024) [27] providing a methodology for quantifying the rate of carbon emissions of software systems [24].

The **goal** of this study is evaluate and optimize the carbon emissions of AFAS SB, a cloud-based enterprise software platform developed by AFAS Software [3] in the Netherlands. Specifically, we firstly apply the SCI methodology [24] to quantify the baseline rate of carbon emissions of AFAS SB in its (Azure-based) production environment. Secondly, we design and conduct a set of in-the-lab experiments for assessing how 18 application-level configurations across 5 categories (*e.g.,* garbage collector, logging, caching, etc.) affect the rate of carbon emissions of AFAS SB across 79 functional test scenarios. Thirdly, we assess how (i) the combination of the application-level configurations impacts the rate of carbon emissions of AFAS SB and (ii) their impact to the overall performance and functionality of the application.

Our **results** reveal distinct effectiveness patterns across optimization strategies, with garbage collection optimization achieving significant carbon reductions of 29.5-30.6%. This pattern reflects AFAS SB's specific system constraints: correlation analysis shows that disk I/O read operations tend to be positively correlated with carbon emissions (r = 0.997), while CPU utilization is negatively correlated (r = -0.82); this suggests that AFAS SB tends to be constrained by disk I/O bottlenecks rather than computational capacity. Garbage collection optimization directly reduces disk I/O read operations because inefficient garbage collection leads to memory allocation patterns that increase disk access for object retrieval and heap management operations. Parallelism configurations that optimize computational throughput tend to have a minimal impact on the actual performance constraints. The combined configurations achieved only 25.7% SCI reduction compared to individual garbage collection optimizations, indicating optimization interference.

The **main contributions** of this study are: (i) definition of a *structured approach for applying the SCI specification* to evaluate carbon emissions in enterprise web applications; (ii) detection of *energy consumption hotspots in a real enterprise web application* (AFAS SB) and their relationship to application-level configurations; (iii) a set of *practical optimization insights* for reducing AFAS SB's carbon footprint while maintaining performance requirements; (iv) an *in-depth discussion* of the implications of our results, including recommendations for researchers and practitioners; (v) a *complete replication package* containing all supporting artifacts and raw data for independent verification [32]; (vi) an *official case study contribution to the Green Software Foundation* (GSF) [2].

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
ICSE-SEIP '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2426-8/2026/04
https://doi.org/10.1145/3786583.3786850

2 Background

2.1 Software Carbon Intensity Specification

The SCI score represents the rate of carbon emissions per functional unit. The SCI score is calculated as $SCI = \frac{O+M}{R}$, where Operational emissions O capture energy-related carbon emissions, while embodied emissions M account for manufacturing and disposal carbon footprint. R represents the functional unit that defines how the application scales. Both operational and embodied emissions are measured in grams of carbon dioxide equivalent (gCO_2eq), a metric that expresses the warming potential of all greenhouse gases in terms of the equivalent amount of CO_2 [22].

Operational emissions O are calculated as $O = E \times I$, where energy consumption E is measured in kWh and includes all energy used by provisioned hardware. This can be measured at datacenter, machine, service, or code execution levels. Carbon intensity I represents the grid carbon intensity in gCO_2eq/kWh. Grid-connected systems use marginal or average emissions factors, while isolated systems require appropriate local emissions factors.

Embodied emissions M are calculated as $M = TE \times \frac{TiR}{EL} \times \frac{RR}{ToR}$, where the Total Embodied emissions TE come from Life Cycle Assessment data of hardware components, measured in gCO_2eq. When detailed LCA data is unavailable, simplified estimation models are used. TiR represents time reserved for the software, while EL indicates the expected hardware lifespan, both measured in the same temporal units. RR quantifies resources reserved for the software, such as CPU cores. In cloud environments, this commonly uses virtual CPU percentages. ToR represents the total available resources in matching units.

Functional units commonly include API calls, users, transactions, database operations, or time-based units.

SCI offers a unified method for quantifying software's carbon impact across contexts, ensuring consistent measurement and informed design decisions. The approach demonstrates universal applicability, accommodating evaluation scenarios ranging from complex distributed systems to individual application components. The methodology emphasizes direct emission reduction as the exclusive mechanism for score improvement, explicitly excluding offset-based strategies.

2.2 AFAS SB: Subject Application

AFAS SB is a cloud-native financial administration platform used for tasks such as invoice processing and financial reporting. Built on .NET 8, the application comprises approximately 4 million lines of C# and 1.8 million lines of TypeScript (including tests) across approximately 30 modules, maintained by a team of 16 developers. In development since 2012 and production since 2020, the platform currently serves over 23,000 companies across more than 500 production environments. As shown in Figure 1, AFAS SB implements a multi-tenant architecture where a single application instance serves multiple customers while maintaining strict data isolation. The application adopts a Command Query Responsibility Segregation (CQRS) design [21], separating read and write operations for performance optimization. AFAS SB is deployed across Microsoft Azure [38] using Azure Service Fabric [37] for orchestration and cluster management. Figure 1 provides a high-level overview of the production environment. Client requests enter through an Azure load balancer (`Clstr-LB`) and are routed to a proxy virtual machine (VM) scale set (`Proxy001`) that handles external-facing communication. These proxy components forward requests to internal Service Fabric clusters (`Clstr-SS`), which provide application lifecycle management and health monitoring capabilities [29]. The SQL elastic pools (`NextTenantPool`) manage database services, sharing computational resources across tenant databases. Supporting infrastructure includes dedicated proxy VMs (`SvcProxy-VM0` and `SvcProxy-VM1`), a logging VM running `Kibana` [17], and an Elastic-VM running `Elasticsearch` [16] for enhanced observability.

Figure 1 shows three additional components: Azure Blob Storage (used for storing tenant attachments), a Login Platform (handling authentication), and Shared Services (supporting payment and banking integrations). The boundary used for the SCI assessment includes only those components responsible for request handling, application execution, and data persistence within Azure: `Clstr-LB`, `Proxy001`, `Clstr-SS`, `NextTenantPool`, `SvcProxy-VM0`, `SvcProxy-VM1`, `Logging-VM`, and `Elastic-VM`.

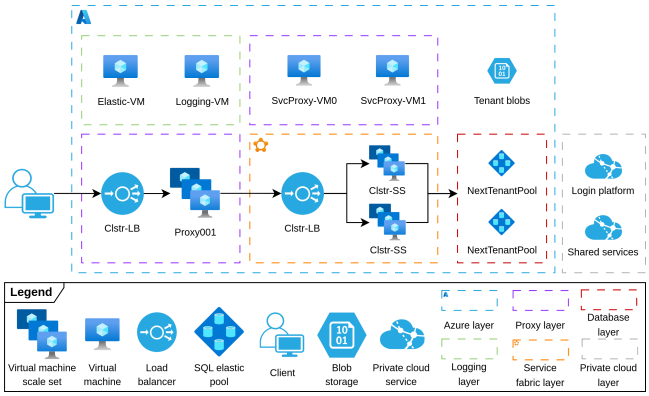

Figure 1: Software architecture of the AFAS SB application

The subject of this experiment is AFAS SB. The selection was based on the application representing a contemporary web application with significant production deployment, providing a relevant context for investigating carbon optimization strategies. The implementation offers extensive configuration parameters across runtime environment variables and application settings that can be manipulated to analyze their impact on energy consumption while maintaining application functionality. The experimental workload consists of 79 test scenarios selected from the application's functional test suite. These scenarios are part of AFAS SB's production testing framework, developed by the test team at AFAS Software using a custom parser with Dutch language support for business process validation. The scenarios are executed routinely by the test team for regression testing and quality assurance during software releases. The test scenarios cover various business processes, including financial administration (31 scenarios), banking and cash management (18 scenarios), tax processing and compliance (12 scenarios), asset management (7 scenarios), and system configuration (11 scenarios). Scenario complexity ranges from simple 3-step operations to complex 89-step business workflows, with an average of 23.4 steps per scenario (median: 19 steps). During experimental execution, scenarios averaged 57.2 seconds duration (median: 20.5 seconds) with a 99.9% success rate across all experimental runs.

The scenario selection focused on core business processes while prioritizing scenarios that require substantial computational effort from the underlying infrastructure.

3 Related Work

Geographic and temporal optimization. Several studies leverage geographic and temporal carbon intensity variations to reduce emissions. For example, `GreenCourier` [9] schedules serverless functions across regions using WattTime [54] data, reducing emissions by 13.25%. Dodge et al. [14] apply similar strategies for AI workloads, achieving up to 30% savings through flexible scheduling during low-carbon periods.

System-Level carbon measurement. At the OS level, `carbond` [49] tracks operational and embodied emissions at the component level, while `cMemento` [31] enables carbon-aware memory placement in Linux. These approaches provide comprehensive accounting but require OS- and hardware-specific interventions. For application-level tracking, `CodeCarbon` [50] is a widely-adopted Python package that estimates CO_2 emissions by monitoring CPU, GPU, and RAM power consumption and combining this with regional carbon intensity data. Similarly, `eco2AI` [8] focuses on accurate energy tracking with an extensive database of regional emission coefficients.

Model-specific carbon assessment. Research on AI workloads applies SCI to compare models and optimize inference. Everman et al. [19] measure CPU/GPU power for GPT models, while Hoffmann and Majuntke [25] route prompts to carbon-efficient models. Bashir et al. [5] identify the "sunk carbon fallacy," showing that misusing embodied emissions in scheduling can increase total emissions by 30%, stressing correct SCI application.

Specialized environment applications. Research has also explored SCI applications in resource-constrained devices. For example, Sayeedi et al. [48] adapt SCI for IoT, incorporating device-specific parameters such as execution time and CPU temperature.

Summary. Existing research makes use of the SCI specification, however, researchers tend to employ *either* (i) cloud-based estimation approaches or (ii) controlled laboratory measurements in isolation. In this work, we address a methodological gap by *combining both approaches*: (i) establishing baseline carbon metrics from a large-scale cloud infrastructure system in production and (ii) conducting controlled experiments with multiple configurations in the lab. Moreover, by evaluating how application-level parameters affect CO2 emissions, we provide evidence-based guidance for reducing the environmental impact of software systems.

3.1 Study Design

3.1.1 Research Questions. This work answers 3 research questions.

RQ1: What is the baseline rate of carbon emissions of AFAS SB in production? Establishing a quantitative baseline is essential for understanding the carbon footprint of AFAS SB in its production environment. This baseline characterizes the overall carbon emissions of the system and identifies which infrastructure components contribute most significantly to total emissions. This question will be answered by applying the SCI methodology to resource utilization data collected from production monitoring systems.

RQ2: How do application-level configuration settings affect the rate of carbon emissions of AFAS SB? Understanding the individual impact of specific configurations enables the development of targeted optimization strategies that software developers can implement without requiring architectural changes. This evaluation identifies the configurations that provide carbon emission reductions and those that have negligible effects. This question will be addressed through controlled experiments determining the SCI score and system performance under different configurations.

RQ3: To what extent can the rate of carbon emissions of AFAS SB be reduced by combining optimal application-level configurations while maintaining system performance and functionality? Moving beyond individual configuration optimization opens the possibility of achieving greater carbon reductions through integrated approaches. This question quantifies the extent of emission reduction possible while maintaining system performance and functionality. This question will be evaluated by determining the combined effect of optimal configurations on the SCI score, execution performance, and functional correctness.

Table 1: Dependent variables

Name	Unit	Description
Baseline SCI Score	gCO_2eq/h	SCI score calculated for production baseline assessment from Azure metrics per hour.
Experimental SCI Score	$gCO_2eq/scenario$	SCI score calculated for the controlled experiments per functional unit (test scenario) enabling direct comparison of equivalent workloads across different configurations.
Power	W	Average power consumption measured by PowerJoular during application execution.
Test Duration	seconds	Time required to execute the entire test suite.
Test Success Rate	%	Percentage of successfully-completed test scenarios.
CPU Utilization	%	Average percentage utilization of CPU resources during experimental run execution.
Disk I/O Read Operations	operations	Total number of disk I/O read operations performed during execution, measured via psutil process monitoring.
Disk I/O Write Operations	operations	Total number of disk I/O write operations performed during execution, measured via psutil process monitoring.

3.1.2 Experimental Variables. Following standard practices [6], experimental variables are defined as follows:

Independent Variables. The independent variables represent the configuration parameters being manipulated in this experiment. The five configuration categories (parallelism, logging, caching, compression, and garbage collection) were identified by reviewing AFAS SB's application-level settings and the .NET runtime configuration options together with the back-end development and Developer Experience teams at AFAS Software. The final selection was based on each category's practical relevance and potential impact on system behavior. Specific parameter settings within each category were informed by existing literature on software energy efficiency. Table 2 lists all independent variables and the value each configuration sets; the replication package [32] contains the complete experimental design with supporting references.

Dependent Variables. These variables represent the outcomes measured during each experimental run to assess the impact of configuration changes. These variables capture quantifiable aspects of system behavior, including energy consumption, resource utilization, performance characteristics, and the SCI score (see Table 1).

Table 2: Independent variables with configuration values (corresponding to Table 5)

Category (ID)	Parameter	Description	Value per Configuration
Parallelism (P2_E1)	`DefaultMaxParallelism`	Sets the maximum degree of parallel task execution.	4 (`parallel_4_lookahead_default`), 16 (`parallel_16_lookahead_default`)
	`DefaultMaxLookAhead`	Determines how many operations can be prepared ahead of execution for improved scheduling.	12 (`parallel_default_lookahead_12`), 48 (`parallel_default_lookahead_48`)
Logging (P2_E2)	`enableTracing`	Enables tracing functionality that tracks request flows.	true (`reduced`), false (`minimal`)
	`consoleLogging`	Controls whether log messages are written to console output during application execution.	false (`reduced`, `minimal`)
	`logLevel`	Defines the minimum severity level for log processing and determines log volume.	Warning (`reduced`), Error (`minimal`)
Caching (P2_E3)	`maxCachedItems`	Sets the maximum number of items that can be stored in the cache simultaneously.	10000 (`extended_expiration`), 20000 (`larger_cache`)
	`expirationInSeconds`	Determines how long cached items remain valid before expiration.	300 (`extended_expiration`), 60 (`larger_cache`)
Compression (P2_E4)	`enableResponseCompression`	Enables compression of HTTP responses.	false (`no_compression`)
Garbage Collection (P2_E5)	`DOTNET_gcServer`	Enables server garbage collection mode with dedicated threads for multi-core environments.	1 (all four configurations)
	`DOTNET_gcConcurrent`	Enables concurrent garbage collection, allowing the collector to run with application threads.	1 (all four configurations)
	`DOTNET_GCConserveMemory`	Sets memory conservation level on a 0-9 scale for collection aggressiveness.	7 (`MemoryConservation`), 5 (`ReducedFootprint`)
	`DOTNET_GCLOHThreshold`	Defines size threshold for Large Object Heap allocation in bytes.	140000 (`MemoryConservation`)
	`DOTNET_GCDynamicAdaptationMode`	Enables dynamic adaptation of garbage collection behavior based on allocation patterns.	1 (`DynamicAdaptation`)
	`DOTNET_GCRetainVM`	Controls whether the garbage collector releases virtual memory back to the operating system.	0 (`DynamicAdaptation`, `ReducedFootprint`)
	`DOTNET_GCNoAffinitize`	Disables processor affinity for garbage collection threads.	1 (`ThreadEfficient`)
	`DOTNET_GCHeapCount`	Limits the number of garbage collection heaps created for memory management.	4 (`ThreadEfficient`)
	`DOTNET_GCHighMemPercent`	Sets memory usage threshold percentage for triggering garbage collection cycles.	46 (`ReducedFootprint`)

3.1.3 Experimental Design. This experiment follows a factorial design [55] with a phased approach to evaluate the impact of different configuration parameters on energy consumption and performance. Each phase targets specific subsets of the independent variables.

Baseline Calculation. This initial phase involves analyzing production metrics from the Azure Service Fabric deployment to establish a baseline SCI score for AFAS SB in its real-world environment. The baseline assessment targets CPU and memory utilization data collected over a one-week monitoring period from the core infrastructure components, including virtual machine scale sets, standalone virtual machines, and SQL elastic pools that comprise the primary production deployment.

Experimental Phases. The controlled experiments are organized into three phases. **Phase 1** establishes the reference point using default values for all independent variables through baseline configuration measurements (P1_E1), serving as the reference point for subsequent comparisons. **Phase 2** employs parameter manipulation across five configuration categories, with each experiment targeting specific independent variable groups. **Phase 3** implements an integrated optimization strategy (P3_E1) combining the most effective parameter settings identified across all variable groups in Phase 2. Further details about the experimental design are available in the replication package. For each experimental condition, 10 repetitions are performed to account for measurement variability.

3.1.4 Data Analysis. Statistical test selection is based on distributional assessment using Shapiro-Wilk tests [51]. Mann-Whitney U tests are primarily employed for two-group comparisons due to non-normal distributions in the experimental data [34]. Effect sizes are calculated using Cohen's d with pooled standard deviation, following conventions of small ($|d| \geq 0.2$), medium ($|d| \geq 0.5$), and large ($|d| \geq 0.8$) effects [13]. Statistical significance is evaluated at the conventional threshold of $\alpha = 0.05$ [20]. Bootstrap confidence intervals provide a robust assessment of effect size estimates [15]. Multiple comparison correction using the Holm method controls family-wise error rate across multiple statistical tests [26]. Post-hoc power analysis assesses the adequacy of sample sizes for detecting observed effect sizes [13]. Correlation analysis using Pearson correlation coefficients [44] examines relationships between resource utilization metrics and carbon intensity measures to identify underlying performance drivers. The analysis approach is tailored to address each research question with specific statistical methods, specifically: (i) for RQ1, baseline carbon emissions characterization through descriptive analysis of production data, including SCI score calculation and service-type comparisons; (ii) for RQ2, configuration effects evaluated through pairwise comparisons against the baseline; (iii) for RQ3, combined optimization assessment comparing the integrated optimal SCI score configurations against the baseline.

3.2 Infrastructure and Experiment Execution

3.2.1 Baseline Calculation Pipeline. To establish the baseline carbon emissions for AFAS SB in production, raw Azure telemetry data

is systematically processed and converted into SCI scores. This baseline calculation is implemented through a multi-stage processing pipeline in Python. Raw CPU and memory metrics are preprocessed to extract instance-specific utilization patterns, followed by data standardization and resampling to consistent 5-minute intervals.

The processed utilization data is fed into the `Impact Framework` [23], a reference implementation for SCI calculations developed by the Green Software Foundation. The framework converts resource utilization metrics into standardized carbon emissions using a manifest that defines the complete calculation pipeline: energy estimation from CPU and memory usage, embodied emissions allocation based on hardware specifications, and final SCI score computation.

3.2.2 Experimental Infrastructure and Implementation. All experiments are conducted in the Green Lab, a research infrastructure at Vrije Universiteit Amsterdam equipped for measuring software energy consumption [33]. The Green Lab uses precise power measurement capabilities, ensuring reliable and repeatable energy measurements. The experiments in this study use a server with an Intel Xeon Silver 4112 processor at 2.60 GHz, 196 GB of RAM, and a `Debian` 9 OS running the `Proxmox` virtualization platform [46]. A two-VM setup is established to separate application execution from test orchestration, ensuring that energy consumption measurements are not affected by test execution overhead. The Application VM runs Windows Server [41] 2022 Datacenter, hosting the AFAS SB application and Microsoft SQL Server [40] 2022. The Test Execution VM also operates Windows Server 2022 Datacenter, running the `Playwright` [36] test automation framework and a PowerShell [39] orchestration script that manages test execution and result processing. Figure 2 illustrates the complete system architecture deployed in the Green Lab environment. The experimental execution is coordinated through `Experiment Runner` [30], an open-source framework that automates experiment configuration, execution management, and data collection across experimental runs. The framework is extended with custom functionality to handle virtual machine communications and manage energy measurement tools.

The architecture employs complementary measurement approaches coordinated through this framework. `PowerJoular` provides process-specific energy consumption measurements while isolating consumption attributable to individual applications, eliminating interference from unrelated system processes [42]. `psutil`, a cross-platform Python library that accesses operating system APIs, delivers system resource metrics including CPU utilization, memory consumption, and I/O operations [47]. The communication infrastructure utilizes distinct protocols for different operational requirements: SSH for parameter configuration and application lifecycle management, SCP for measurement data collection, and HTTP for test execution communication between the Test Execution VM and Application VM. Each experimental run follows a protocol that begins with environment reset procedures, including virtual machine state restoration to eliminate residual processes, SQL Server database restoration to standardize content and schema, and system cache clearing to ensure consistent initial conditions between experimental treatments. The experimental treatments are implemented through two configuration approaches targeting the different independent variable categories. Parallelism and garbage collection settings are controlled through environment variables that are applied before the application startup. Logging, caching, and compression configurations are modified through the application's configuration file, with parameter values updated according to the experimental design specifications. The complete replication package and dataset are publicly available on GitHub [32].

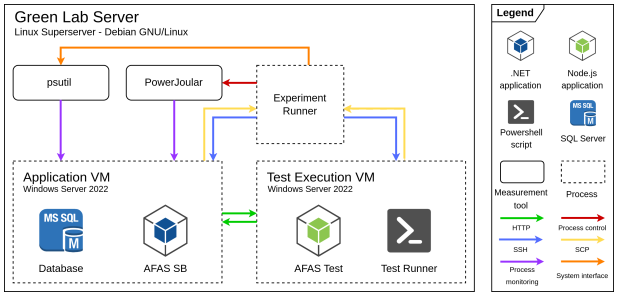

Figure 2: Experimental setup architecture

3.2.3 Carbon Emissions Calculation Methodology. The SCI calculation uses distinct computational approaches for Azure production data (RQ1) and Green Lab experimental data (RQ2-RQ3), both implementing the same core methodology with different parameter values and calculation processes.

Azure Production SCI Calculation Pipeline. The Azure baseline calculation uses the `Impact Framework` with a manifest-defined pipeline that processes resource utilization data through sequential computational steps. This pipeline approach enables grouping by instance type and aggregating results across the heterogeneous Azure infrastructure with multiple instance types and configurations:

CPU Energy Calculation. CPU utilization percentages are converted to power consumption using linear interpolation between utilization points [0, 10, 50, 100%] and corresponding power factors [0.12, 0.32, 0.75, 1.02]. The resulting CPU energy calculation: $E_{cpu} = \frac{\text{CPU_factor} \times \text{TDP} \times \text{duration_seconds}}{1000 \times 3600}$ where CPU_factor represents the interpolated power coefficient based on utilization, TDP is the thermal design power for each instance type (e.g., 280W for `Standard_E16ads_v5`), and duration_seconds is the time interval represented by each Azure monitoring data point.

Memory Energy Calculation. Memory energy uses a fixed coefficient of 0.000392 watts per GB [12]: $E_{memory} = \text{memory_GB} \times 0.000392 \times \frac{\text{duration_seconds}}{3600}$ where memory_GB represents the total memory allocation for each instance.

Embodied Emissions Calculation. Using a 4-year hardware lifespan (126,144,000 seconds): $M = \text{TE} \times 1000 \times \frac{\text{duration_seconds}}{126144000} \times \frac{\text{vCPUs_allocated}}{\text{total_server_cores}}$ where TE represents total embodied emissions from Cloud Carbon Footprint coefficients (e.g., 2010.54 kg CO_2eq for `Standard_E16ads_v5`), vCPUs_allocated is the number of virtual CPUs assigned to each instance, and total_server_cores represents the total number of vCPUs that can be allocated on the underlying Azure instance type.

Final SCI Computation. Converting duration to hours for the functional unit: $\text{SCI} = \frac{(E_{cpu}+E_{memory}) \times 355 + M}{\text{duration_seconds}/3600}$ using 355 gCO_2eq/kWh grid carbon intensity for The Netherlands, representing the average value from ElectricityMaps [18] for the baseline measurement week (February 3-9, 2025).

Green Lab Experimental SCI Calculation. The experimental SCI calculation implements direct computational steps using measured power consumption data. Unlike the baseline assessment, all experiments use identical hardware specifications, eliminating the need for instance-type grouping and enabling direct calculation:
Energy Conversion. Measured power consumption is converted to energy: $E = \frac{\text{power_watts} \times \text{duration_seconds}}{1000 \times 3600}$ where power_watts represents the average power consumption measured by `PowerJoular` during test execution.
Operational Emissions. Using grid carbon intensity: $O = E \times 272$ applying 272 gCO_2eq/kWh grid carbon intensity for The Netherlands, representing the monthly average from ElectricityMaps [18] for April 2025 when the controlled experiments were conducted.
Embodied Emissions. Using the same 4-year hardware lifespan (35,040 hours): $M = 1449.84 \times 1000 \times \frac{\text{duration_seconds}}{126144000} \times \frac{8}{24}$ using Green Lab server specs with 1449.84 kg total embodied emissions and 8-core allocation representing the Application VM's vCPU allocation from the total 24 VM cores allocated across the experimental setup (8 cores for Application VM + 16 cores for Test Execution VM).
Final SCI Computation. Using test scenarios as the functional unit: $\text{SCI} = \frac{E \times 272 + M}{\text{scenario_count}}$ where scenario_count represents the number of functional test scenarios executed during each experimental run.

The embodied emissions component utilizes the Cloud Carbon Footprint (CCF) coefficients methodology [11], an open-source framework that enables the calculation of Total Embodied Emissions values for cloud infrastructure components. The CCF methodology distributes hardware manufacturing emissions across expected component lifetime, accounting for CPU architecture, memory configuration, and storage specifications.

For the baseline calculations addressing RQ1, Azure instance specifications were collected through direct inspection of resources in Azure, with detailed hardware specifications obtained from CloudPrice [1]. These specifications were integrated into the CCF coefficients repository to enable the calculation of TE values for the Azure production environment. For the controlled experiments addressing RQ2 and RQ3, separate TE values were calculated using the Green Lab server hardware specifications.

4 Results

4.1 Baseline Assessment (RQ1)

4.1.1 Production Environment Characterization. The baseline assessment analyzed 46 virtual machine instances across 12 resource groups over a one-week monitoring period in February 2025. The production environment exhibited substantial performance heterogeneity, with a system-wide mean SCI score of **14.162 gCO_2eq/h** per infrastructure instance (SD = 11.011, n = 46). Table 3 presents the system-level carbon performance characteristics.

The high coefficient of variation (70.8%) reflects the heterogeneous nature of the production deployment, which includes different infrastructure types with distinct operational roles. Scale sets serve various functions, with some acting as proxy servers while others host the core application. Workload distribution varies across clusters based on demand patterns.

4.1.2 Service Type Performance Distribution. Analysis of carbon performance by service type reveals distinct efficiency patterns across the AFAS SB infrastructure. Table 4 presents carbon performance characteristics categorized by Azure component types.

Table 3: Baseline system-wide carbon performance metrics for AFAS SB production environment

Metric	Value	Unit
Mean SCI Score	14.162	gCO_2eq/instance-hour
Standard Deviation	11.011	gCO_2eq/h
Median SCI Score	12.625	gCO_2eq/h
Coefficient of Variation	70.8%	–
Total Energy Consumption	216.78	kWh
Total Carbon Emissions	109,446.11	gCO_2eq
Sample Size	46	instances

Table 4: Carbon performance characteristics by service type in AFAS SB production deployment

Service Type	Instances	Mean SCI (gCO_2eq/h)	SCI Range (gCO_2eq/h)	Carbon Share (%)
VM Scale Set	30	18.257	1.973 – 33.221	74.2
SQL Elastic Pool	12	10.902	5.691 – 29.844	20.1
Virtual Machine	4	9.295	1.939 – 22.361	5.7

VM Scale Sets, representing 65.2% of instances, account for 74.2% of total carbon emissions with the highest mean SCI score (18.257 gCO_2eq/h). Despite having the lowest instance count, SQL Elastic Pools contribute 20.1% of emissions, indicating higher per-instance carbon intensity. The substantial performance ranges within each service type (e.g., 1.973–33.221 gCO_2eq/h for VM Scale Sets) demonstrate heterogeneous efficiency characteristics even within deployment categories.

4.1.3 Carbon Intensity Distribution Characteristics. The computed SCI score ranges from a minimum of 1.938 gCO_2eq/h and a maximum of 33.221 gCO_2eq/h, with a median of 12.625 gCO_2eq/h. The substantial interquartile range (20.381 gCO_2eq/h) and the observation that 90% of instances fall below 32.903 gCO_2eq/h while the maximum reaches 33.221 gCO_2eq/h suggest that a small number of high-impact instances drive overall system carbon intensity.

4.1.4 Carbon Intensity Visualization. Figure 3 displays the hierarchical SCI score distribution, where rectangle sizes represent relative carbon contributions across service types and resource groups. The treemap visualization confirms the concentration of carbon emissions within specific infrastructure components, with `Cluster004-RG` and `Cluster003-RG` dominating the carbon footprint despite representing a minority of total instances.

4.2 Configuration Impact Analysis (RQ2)

For RQ2, a controlled experiment is conducted to compare configurations against the baseline SCI score of 0.457 gCO_2eq/scenario (±0.031, n = 10), established in the Green Lab environment.

4.2.1 Configuration Performance Analysis. Table 5 presents the complete performance characterization across all experimental configurations, including statistical validation and effect size measurements. Notable values indicating significant improvements are highlighted in bold. We identify distinct performance patterns across configuration categories. All garbage collection configurations (P2_E5) achieved SCI scores below 0.323 gCO_2eq/scenario,

representing 29.5–30.6% improvements over baseline. These configurations also demonstrated the largest effect sizes (4.75–5.25) and achieved perfect statistical power (1.0). In contrast, parallelism configurations (P2_E1) showed modest improvements ranging from 0.015% to 5.3%, with the parallel_16_lookahead_default configuration achieving the best performance (0.433 gCO_2eq/scenario). However, these improvements did not reach statistical significance after p-value correction, with an achieved power of 0.24.

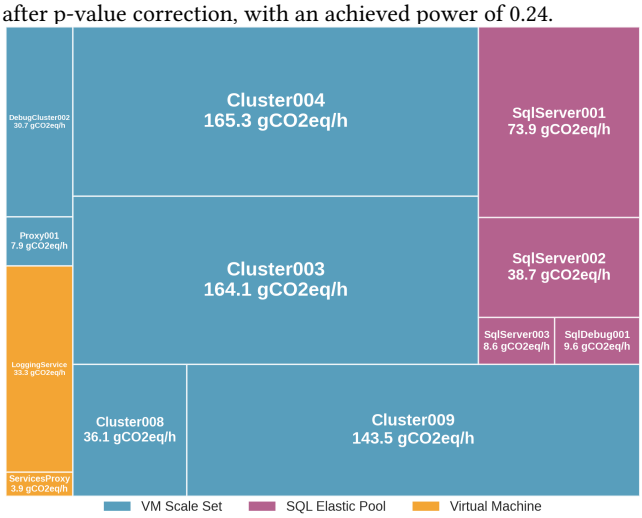

Figure 3: Carbon intensity by service type and resource group

4.2.2 Statistical Significance and Multiple Comparison Analysis. Six configurations have statistically significant SCI score reductions at the conventional $\alpha = 0.05$ level before correction: `P2_E2_minimal` ($p = 0.045$), `P2_E3_extended_expiration` ($p = 0.021$), and all four garbage collection configurations (`P2_E5_MemoryConservation`, `P2_E5_DynamicAdaptation`, `P2_E5_ThreadEfficient`, `P2_E5_ReducedFootprint`) with p-values of 0.00018. However, after applying the Holm correction for multiple comparisons, only four configurations maintained statistical significance, with all four configurations achieving Holm-corrected p-values of 0.010. The configurations with robust statistical evidence after correction were exclusively from the garbage collection optimization category (P2_E5), demonstrating considerable practical significance according to Cohen's conventions. Bootstrap confidence intervals confirm the robustness of these findings, with all significant configurations maintaining confidence intervals that exclude zero effect. The `P2_E5_DynamicAdaptation` configuration achieved the strongest statistical evidence with a bootstrap confidence interval of [4.03, 48.20]. The wide upper bound reflects bimodal execution time patterns (8 measurements at 3470-3500s, 2 at 4070-4090s) characteristic of system testing variability.

4.2.3 Performance Distribution Visualization. Figure 4 shows SCI score distributions across configuration categories. We observe a clear separation between baseline performance and garbage collection optimizations, with no overlap between the baseline and the P2_E5 configuration family. The tight distributions within each garbage collection configuration (standard deviations 0.021–0.025) indicate consistent performance improvements across repetitions. For other configuration categories, the distributions overlap substantially with baseline performance, consistent with the statistical analysis, which shows non-significant improvements. The parallelism configurations show slight leftward shifts (lower SCI scores) but with considerable overlap with baseline measurements.

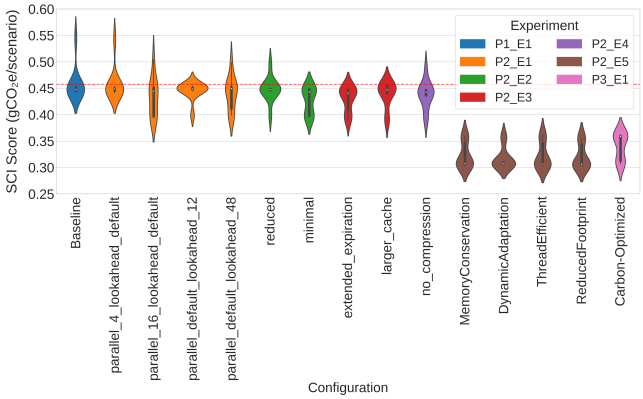

Figure 4: SCI scores across configuration categories, with the red dotted line showing the mean SCI score of the baseline

4.2.4 Multi-Dimensional Performance Analysis. Figure 5 presents improvement percentages and effect sizes across multiple performance metrics, providing a complete view of configuration impacts beyond SCI scores. The heatmap analysis reveals that carbon improvements are consistently accompanied by execution time reductions, with garbage collection configurations achieving 29.0–30.3% duration improvements alongside their carbon benefits. This pattern indicates that optimization mechanisms address fundamental system bottlenecks rather than creating isolated improvements. Power consumption improvements vary across configurations, with caching and compression optimizations achieving 2.0–2.7% reductions while garbage collection configurations show more modest gains (1.2–1.5%). This pattern suggests that for the configurations tested in this study, carbon benefits derive primarily from execution time efficiency rather than direct energy reduction. CPU utilization patterns exhibit mixed responses across configurations, indicating distinct optimization pathways within the tested categories.

4.2.5 Resource Utilization Correlation Analysis. Figure 6 displays correlation patterns between resource metrics and SCI scores across all tested configurations. We observe a strong negative correlation between CPU utilization and SCI scores ($r = -0.82$, $p < 0.001$), indicating that higher CPU utilization tends to correspond to lower carbon emissions per functional unit within the tested configurations. This counterintuitive relationship is primarily driven by shorter execution times: configurations achieving higher CPU utilization (such as garbage collection optimizations) execute the test suite in 3604-3670 seconds compared to the baseline's 5169 seconds, representing 29.0-30.3% duration reductions alongside their carbon improvements. Disk I/O read operations demonstrate a near-perfect positive correlation with SCI scores ($r = 0.997$, $p < 0.001$), indicating that carbon emissions tend to be strongly associated with the frequency of disk access operations across the experimental conditions.

4.2.6 Hypothesis Testing Results for RQ2. Four configurations from the garbage collection category maintain statistical significance under conservative Holm correction, with all four achieving Holm-corrected p-values of 0.010. The effect sizes for significant configurations (4.75 – 5.25) indicate considerable practical relevance.

Table 5: Configuration analysis with performance metrics and statistical validation

Configuration	SCI Score (gCO_2eq/scenario) Mean	Std	Min	Max	Median	Duration (sec)	Power (W)	Success Rate (%)	Effect Size	Bootstrap CI (95%)	Original p-value	Holm p-value	Achieved Power
P1_E1 Baseline	0.457	0.031	0.442	0.546	0.447	5169	41.81	99.74	–	–	–	–	–
P2_E1_parallel_4_lookahead_default	0.457	0.031	0.441	0.544	0.448	5172	41.73	99.74	0.00	-0.86 – 0.86	0.970	1.0	0.24
P2_E1_parallel_16_lookahead_default	0.433	0.036	0.394	0.504	0.445	4885	42.03	99.87	0.71	-0.12 – 1.68	0.385	1.0	0.24
P2_E1_parallel_default_lookahead_12	0.445	0.017	0.399	0.460	0.449	5022	41.98	99.87	0.48	-1.11 – 1.01	0.678	1.0	0.24
P2_E1_parallel_default_lookahead_48	0.440	0.032	0.399	0.498	0.449	4959	42.03	100.0	0.56	-0.40 – 1.37	0.970	1.0	0.24
P2_E2_reduced	0.448	0.023	0.398	0.496	0.448	5056	41.94	100.0	0.34	-0.67 – 1.00	0.970	1.0	0.52
P2_E2_minimal	0.427	0.025	0.395	0.449	0.443	4806	42.19	100.0	1.06	0.59 – 1.96	0.045	1.0	0.52
P2_E3_extended_expiration	0.430	0.024	0.390	0.450	0.441	4902	40.96	99.87	0.99	0.59 – 1.71	0.021	0.63	0.69
P2_E3_larger_cache	0.438	0.025	0.388	0.459	0.447	5014	40.67	100.0	0.67	-0.32 – 1.25	0.678	1.0	0.69
P2_E4_no_compression	0.438	0.027	0.390	0.487	0.443	5004	40.76	99.87	0.68	-0.11 – 1.39	0.054	1.0	0.48
P2_E5_MemoryConservation	**0.322**	0.024	0.304	0.359	0.307	**3670**	41.19	100.0	**4.79**	**3.83 – 11.70**	**0.00018**	**0.010**	**1.0**
P2_E5_DynamicAdaptation	**0.317**	0.021	0.306	0.357	0.307	**3604**	41.33	100.0	**5.25**	**4.03 – 48.20**	**0.00018**	**0.010**	**1.0**
P2_E5_ThreadEfficient	**0.322**	0.025	0.303	0.361	0.306	**3663**	41.27	99.74	**4.75**	**3.80 – 11.36**	**0.00018**	**0.010**	**1.0**
P2_E5_ReducedFootprint	**0.320**	0.024	0.303	0.355	0.306	**3646**	41.17	100.0	**4.91**	**3.92 – 12.02**	**0.00018**	**0.010**	**1.0**
P3_E1_Carbon-Optimized	**0.340**	0.026	0.307	0.362	0.359	**3866**	41.26	100.0	**4.10**	**3.52 – 7.55**	**0.00018**	**0.010**	**1.0**

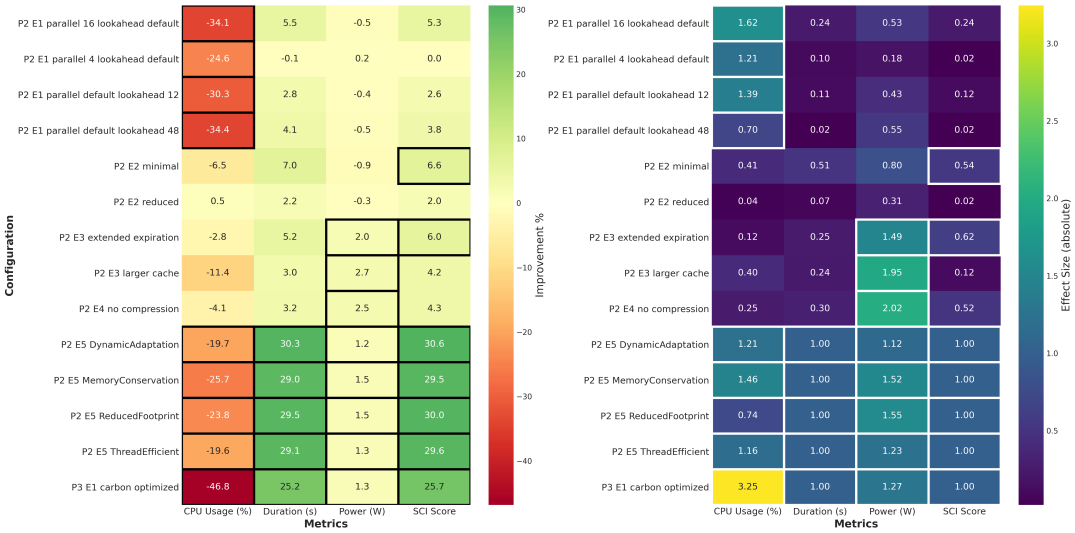

Figure 5: Configuration impact heatmap showing improvement percentages (left) and effect sizes (right) across performance metrics. Black borders indicate statistical significance for improvements, white borders indicate significance for effect sizes.

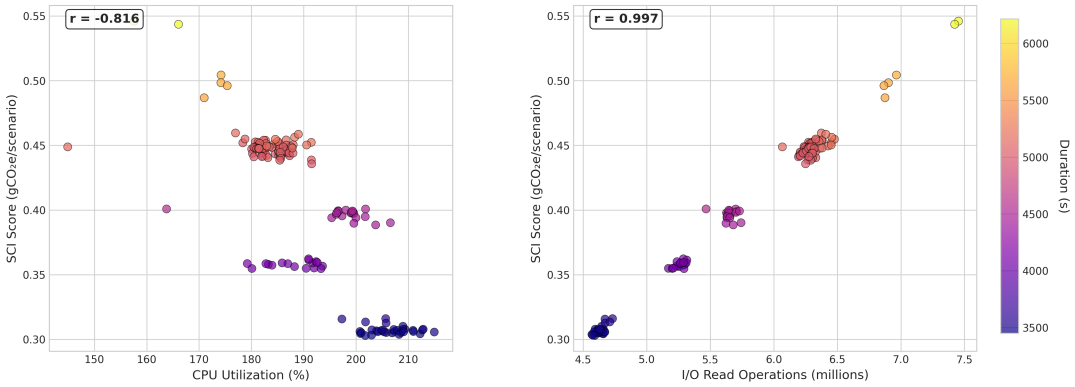

Figure 6: Resource utilization patterns vs SCI scores showing correlation relationships: CPU utilization (r = -0.82, p < 0.001) and disk I/O read operations (r = 0.997, p < 0.001) colored by test duration

4.3 Combined Configuration Analysis (RQ3)

4.3.1 Configuration Integration. The CO2-optimized configuration combines parameter settings from the best-performing configuration in each Phase 2 category, provided they achieve at least 5% SCI score reduction wrt the baseline. The integrated approach combines the `DynamicAdaptation` garbage collection (30.6% reduction, best in category), parallel_16_lookahead_default parallelism (5.3% reduction, best in category), minimal logging (6.6% reduction, best in category), and extended_expiration caching configurations (5.9% reduction, best in category). The compression category's best performer (no_compression, 4.2% reduction) was excluded as it fell below the 5% inclusion threshold.

4.3.2 Integrated Configuration Performance. The carbon-optimized configuration achieved a mean SCI score of 0.340 gCO_2eq/scenario (±0.026, n = 10). The baseline performance was 0.457 gCO_2eq/scenario (±0.031, n=10). This represents a 25.7% reduction in SCI score while maintaining 100% functional correctness across all test scenarios. Mann-Whitney U test yielded U = 0.0 (p = 0.010), indicating complete distributional separation. No overlap occurred between the measurement ranges of the two conditions, with the optimized configuration's maximum value (0.362 gCO_2eq/scenario) falling below the baseline minimum value (0.442 gCO_2eq/scenario).

4.3.3 Statistical Validation and Effect Size. The CO2-aware configuration demonstrates strong statistical evidence with a Holm-corrected p-value of 0.010. Cohen's d effect size of 4.10 indicates considerable practical significance, with the bootstrap confidence interval [3.52, 7.55] for the effect size. The mean difference between baseline (0.457) and optimized (0.340) configurations yields an absolute SCI reduction of 0.117 gCO_2eq/scenario. A 95% confidence interval for this mean difference [0.092, 0.142] gCO_2eq/scenario indicates that the actual improvement lies between 20.1% and 31.1% reduction with 95% probability.

4.3.4 Hypothesis Testing Results for RQ3. The carbon-optimized strategy maintains functional reliability while achieving substantial improvements in SCI score and performance. The effect size (Cohen's d = 4.10) indicates considerable practical significance.

5 Wrap Up and Lessons Learned

Performance Drivers and Configuration Effectiveness. This study demonstrates that application-level configuration optimization achieved substantial SCI score reductions in AFAS SB, with empirical evidence revealing distinct effectiveness patterns across configuration categories. Correlation analysis reveals that disk I/O read operations tend to be a significant factor associated with CO_2 emissions (r = 0.997), with CPU utilization being negatively correlated (r = -0.82), indicating that disk I/O activity tends to be associated with CO_2 efficiency across configuration categories. Garbage collection optimization appears most effective because it may address this significant associated factor: inefficient garbage collection tends to create allocation patterns that may increase disk I/O read operations for object access, heap traversal, and memory management operations. The garbage collection configurations all target different aspects of heap management that may reduce these disk access patterns. Garbage collection optimizations tend to reduce disk I/O read operations and simultaneously improve both execution time (29.0–30.3% reduction) and CO_2 efficiency. The consistency of substantial improvements across four distinct garbage collection strategies aligns with existing research, which shows that application performance improvements can be achieved through various garbage collection optimization techniques, such as memory management optimization [10], heap management tuning [7], and collector strategy selection [45]. While the improvements in this study primarily derive from performance gains that reduce total execution time, research also demonstrates that garbage collector selection can achieve energy reductions of up to 47% [52]. This confirms that garbage collection optimization offers multiple pathways for CO_2 reduction in managed runtime environments.

Parallelism configurations optimize computational throughput, but AFAS SB was constrained by disk I/O operations, rather than CPU capacity, which explains its limited effectiveness. Other configurations similarly failed to address the system's primary bottlenecks, resulting in negligible changes in their resulting SCI scores. The consistent correlation between execution time reduction and SCI score improvement across effective configurations indicates that faster execution was achieved without proportional increases in power consumption, demonstrating efficiency gains per functional unit rather than mere speed optimization. This pattern aligns with research showing that performance improvements can correspond with genuine efficiency gains [**?**]. The garbage collection optimizations therefore achieved genuine efficiency improvements, delivering both faster execution and improved resource utilization per functional unit.

Integration Strategy Results. The CO_2-optimized configuration achieved 0.340 gCO_2eq/scenario (25.7% improvement) but performed worse than individual garbage collection configurations (0.317-0.322), indicating optimization interference rather than additive benefits. This suggests that additional optimizations can disrupt the effectiveness of garbage collection, potentially through altered resource allocation or conflicting bottlenecks. The integrated configuration maintained the pattern observed in individual garbage collection optimizations: simultaneous CO_2 reduction and execution time improvement (25.2%). This demonstrates that in the AFAS SB case study, the environmental and performance benefits consistently occur *together* across both separate and combined optimization approaches. The achievement of substantial CO_2 reduction while maintaining 100% functional correctness indicates that garbage collection optimization delivers environmental improvements without compromising application functionality within the tested scenarios.

Production Environment Implications. The baseline assessment reveals that VM Scale Sets contribute 74.2% of total CO_2 emissions in the AFAS SB production deployment, representing the infrastructure components that host the application instances optimized in this study. This finding provides context for interpreting the controlled experiment results: the application-level configuration optimizations achieved SCI score reductions of up to 30.6% for the AFAS SB application software that runs on these dominant emission sources. While the experimental environment differs from production complexity, the results indicate optimization potential for the software deployed on the infrastructure components responsible for the majority of production CO_2 emissions. The CO_2 optimization approach demonstrated in this study operates within existing VM infrastructure, focusing on software configuration parameters. This complements infrastructure-level optimization

strategies, such as those shown in CO_2-aware VM management frameworks, which achieve CO_2 reductions through rightsizing underutilized instances (e.g., 48% savings by downsizing VM specifications) and regional migration to areas with lower grid CO_2 intensity [43]. For AFAS SB's production deployment, such infrastructure-level analysis could identify instances with suboptimal resource allocation, while the validated application-level optimizations address software efficiency within the allocated resources. When presented with the findings of this study, AFAS expressed interest in implementing garbage collection optimizations, viewing them as easily implementable with potentially high benefits.

Interpretation Limitations. The in-the-lab environment differs significantly from the in-production one, which might limit confidence in the observed results. Nevertheless, bootstrap confidence intervals reflect genuine experimental variability, with bimodal performance patterns in several garbage collection configurations producing wide confidence bounds while maintaining statistical significance. Garbage collection optimization showed consistent benefits under standardized conditions. However, production environments involve workload variability and resource contention that may influence the relative effectiveness of different configurations. The architectural specificity of the findings constrains the applicability of the effectiveness hierarchy observed across configuration categories. The identification of garbage collection optimization as the dominant approach reflects AFAS SB's specific system characteristics, and may not generalize to other applications.

Statistical Rigor and Multiple Comparison Considerations. The experimental design involved 56 statistical comparisons across configuration categories, substantially increasing Type I error probability without multiple comparison correction. The application of the Holm correction for multiple comparisons provides conservative statistical control but may obscure genuine configuration effects with smaller effect sizes. The achievement of statistical significance for garbage collection configurations under this conservative approach strengthens confidence in these findings. The low statistical power achieved for non-garbage collection categories (0.24–0.69) indicates insufficient sample sizes to detect smaller effect sizes reliably. This limitation suggests that configurations showing modest improvements may require larger sample sizes or more sensitive measurement approaches to establish statistically significant evidence of their effectiveness.

Implications for CO_2 Optimization Research. Correlation analysis effectively identified system factors driving emissions, with I/O operations showing a strong correlation ($r = 0.997$). This suggests that resource utilization analysis can guide optimization prioritization and indicates that researchers should investigate opportunities for simultaneous performance and CO_2 improvements, rather than assuming inherent trade-offs.

Experience and Recommendations for SCI Implementation. *Primary Implementation Challenge: Telemetry-to-Energy Conversion.* The most significant challenge encountered was converting cloud telemetry data to energy consumption estimates. Azure does not provide direct energy metrics, requiring estimation approaches using CPU utilization, memory usage, and other resource metrics. Each infrastructure component type presents different available metrics. VM Scale Sets provide CPU and memory utilization percentages, while SQL Elastic Pools report percentage-based metrics that require conversion to absolute values. Standalone VMs, on the other hand, offer direct memory usage in bytes. This heterogeneity necessitates component-specific processing pipelines, which prevents the use of standardized energy calculation approaches across the entire infrastructure.

Methodological Gaps in Current Standards. Organizations implementing SCI should anticipate substantial preprocessing efforts to make cloud telemetry data suitable for energy calculation. Current SCI guidance offers a mathematical framework but lacks practical advice on handling inconsistencies and variations in cloud provider data, as well as metric availability. The absence of standardized telemetry-to-energy conversion methodologies forces practitioners to develop custom solutions, reducing comparability across different implementations and cloud providers.

Recommendations for Standard Enhancement. The Green Software Foundation and ISO/IEC should develop standardized conversion protocols for the telemetry formats of major cloud providers. These protocols should address variations in metric availability across service types and provide validated power modeling coefficients for different instance families. Additionally, guidance on handling missing or incomplete telemetry data would improve implementation reliability, as cloud monitoring systems may not provide consistent metric coverage across all infrastructure components.

Implementation Recommendations for Organizations. Organizations should begin by cataloging available telemetry metrics across their infrastructure before attempting SCI implementation. Different cloud services provide metrics at different granularity and formats, requiring custom preprocessing. Practitioners should also plan for substantial effort in data pipeline development, as cloud monitoring APIs require translation steps to produce SCI-compatible input formats. Finally, organizations should recognize that estimation-based approaches introduce uncertainty that should be acknowledged when communicating CO_2 emissions results to stakeholders.

Common Implementation Pitfalls. The primary pitfall is underestimating the complexity of telemetry data preprocessing. Cloud providers' monitoring systems are designed for operational purposes rather than energy assessment, requiring significant transformation effort. Additionally, organizations should avoid assuming that cloud resource utilization metrics directly correlate with energy consumption patterns, as the relationship depends on hardware characteristics and power management features not visible through standard monitoring interfaces.

6 Conclusion

Application-level configuration optimizations significantly reduced the SCI score of AFAS SB without affecting performance. Garbage collection tuning was most effective, reducing the SCI score by 29.5–30.6% ($p < 0.001$), while combined optimizations achieved 25.7% reduction with full functional correctness. Correlation analysis linked I/O operations ($r = 0.997$) and CPU constraints ($r = -0.82$) to emissions, guiding prioritization. Memory management showed large effects, whereas other categories had minimal impact. Baseline analysis revealed VM Scale Sets account for 74.2% of emissions, highlighting system-wide optimization potential. These results demonstrate evidence-based configuration as a viable path for enterprise software CO_2 reduction.

References

[1] 2024. CloudPrice.net – Cloud Instance Pricing and Specifications. https://cloudprice.net Accessed: 2025-02-24.

[2] 2025. AFAS SB Enterprise Software Application Carbon Assessment - Green Software Foundation SCI case study. https://github.com/Green-Software-Foundation/sci-guide/blob/dev/use-case-submissions/AFAS_case_study.md [Online; accessed 7. Aug. 2025].

[3] AFAS Software B.V. 2025. AFAS Software: ERP and Business Automation Solutions. https://www.afas.nl Dutch family-owned company founded in 1996, Leusden, Netherlands.

[4] Lars Andringa, Brian Setz, and Vasilios Andrikopoulos. 2025. Understanding the Energy Consumption of Cloud-native Software Systems. In *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE '25)*. ACM, London, United Kingdom, 309–319.

[5] Noman Bashir, Varun Gohil, Anagha Belavadi, Mohammad Shahrad, David Irwin, Elsa Olivetti, and Christina Delimitrou. 2024. The Sunk Carbon Fallacy: Rethinking Carbon Footprint Metrics for Effective Carbon-Aware Scheduling. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '24)*. ACM, Redmond, WA, USA, 542–551.

[6] Victor R. Basili, Forrest Shull, and Filippo Lanubile. 1999. Building Knowledge through Families of Experiments. *IEEE Transactions on Software Engineering* 25, 4 (1999), 456–473. https://doi.org/10.1109/32.799939

[7] Tim Brecht, Eshrat Arjomandi, Chang Li, and Hang Pham. 2001. Controlling Garbage Collection and Heap Growth to Reduce the Execution Time of Java Applications. In *Proceedings of the 16th ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications (OOPSLA '01)*. ACM, Tampa Bay, FL, USA, 353–366.

[8] Semen A. Budennyy, Vladimir D. Lazarev, Nikita N. Zakharenko, Alexey N. Korovin, Olga A. Plosskaya, Denis V. Dimitrov, Vladimir S. Akhripkin, Ivan V. Pavlov, Ivan V. Oseledets, Ivan S. Barsola, Ilya V. Egorov, Aleksandra A. Kosterina, and Leonid E. Zhukov. 2023. eco2AI: Carbon Emissions Tracking of Machine Learning Models as the First Step Towards Sustainable AI. *Doklady Mathematics* 106, Suppl 1 (2023), S118–S128. https://doi.org/10.1134/S1064562422060230

[9] Mohak Chadha, Thandayuthapani Subramanian, Eishi Arima, Michael Gerndt, Martin Schulz, and Osama Abboud. 2023. GreenCourier: Carbon-Aware Scheduling for Serverless Functions. In *Proceedings of the 9th International Workshop on Serverless Computing (WoSC '23)*. ACM, Koblenz, Germany, 1–7.

[10] Wen-ke Chen, Sanjay Bhansali, Trishul Chilimbi, Xiaofeng Gao, and Weihaw Chuang. 2006. Profile-guided Proactive Garbage Collection for Locality Optimization. In *Proceedings of the 2006 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '06)*. ACM, Ottawa, Ontario, Canada.

[11] Cloud Carbon Footprint. 2024. Cloud Carbon Footprint Coefficients. https://github.com/cloud-carbon-footprint/cloud-carbon-coefficients Accessed: 2025-02-24.

[12] Cloud Carbon Footprint. 2024. Methodology. https://www.cloudcarbonfootprint.org/docs/methodology/

[13] Jacob Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Erlbaum Associates.

[14] Jesse Dodge, Taylor Prewitt, Remi Tachet Des Combes, Erika Odmark, Roy Schwartz, Emma Strubell, Alexandra Sasha Luccioni, Noah A. Smith, Nicole DeCario, and Will Buchanan. 2022. Measuring the Carbon Intensity of AI in Cloud Instances. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT '22)*. ACM, Seoul, Republic of Korea, 1877–1894.

[15] Bradley Efron and Robert Tibshirani. 1986. Bootstrap Methods for Standard Errors, Confidence Intervals, and Other Measures of Statistical Accuracy. *Statist. Sci.* 1, 1 (1986), 54–75. https://doi.org/10.1214/ss/1177013815

[16] Elastic N.V. 2025. Elasticsearch: The Official Distributed Search and Analytics Engine. https://www.elastic.co/elasticsearch/. Accessed: 2025-06-28.

[17] Elastic N.V. 2025. Kibana: Your Window into the Elastic Stack. https://www.elastic.co/kibana/. Accessed: 2025-06-28.

[18] ElectricityMaps. 2025. ElectricityMaps: Real-time and Historical Electricity Data. https://www.electricitymaps.com/ Accessed: 2025-05-28.

[19] Brad Everman, Trevor Villwock, Dayuan Chen, Noe Soto, Oliver Zhang, and Ziliang Zong. 2023. Evaluating the Carbon Impact of Large Language Models at the Inference Stage. In *Proceedings of the IEEE International Conference on Performance Computing and Communications (IPCCC '23)*. IEEE. https://doi.org/10.1109/IPCCC59175.2023.10253886

[20] Ronald A. Fisher. 1925. *Statistical Methods for Research Workers* (1st ed.). Oliver & Boyd, Edinburgh, Scotland. https://archive.org/details/statisticalmeth00fish Foundational text introducing the 0.05 significance level criterion in statistical testing..

[21] Martin Fowler. 2011. CQRS. https://martinfowler.com/bliki/CQRS.html Technical article on Command Query Responsibility Segregation pattern.

[22] L. K. Gohar and K. P. Shine. 2007. Equivalent CO2 and its use in understanding the climate effects of increased greenhouse gas concentrations. *Weather* 62, 11 (2007), 307–311. https://doi.org/10.1002/wea.103

[23] Green Software Foundation. 2024. *Impact Framework: A framework to Model, Measure, siMulate and Monitor the environmental impacts of software.* Green Software Foundation. https://if.greensoftware.foundation/ Accessed: 2025-05-15.

[24] Green Software Foundation Standards Working Group. 2024. *Software Carbon Intensity (SCI) Specification.* Green Software Foundation. https://sci.greensoftware.foundation Version 1.1.0.

[25] Geerd-Dietger Hoffmann and Verena Majuntke. 2024. Improving Carbon Emissions of Federated Large Language Model Inference through Classification of Task-Specificity. In *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon '24)*. ACM, Santa Cruz, CA, USA, 1–7.

[26] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[27] International Organization for Standardization. 2024. *ISO/IEC 21031:2024 Information technology – Sustainability – Software Carbon Intensity.* International Standard ISO/IEC 21031:2024. ISO, Geneva, Switzerland. https://www.iso.org/standard/86612.html

[28] Jung-Sing Jwo, Jing-Yu Wang, Chun-Hao Huang, Shyh-Jon Two, and Hsu-Cheng Hsu. 2011. An Energy Consumption Model for Enterprise Applications. In *Proceedings of the 2011 IEEE/ACM International Conference on Green Computing and Communications (GreenCom '11)*. IEEE, 216–219. https://doi.org/10.1109/GreenCom.2011.42

[29] Gopal Kakivaya, Lu Xun, Richard Hasha, Shegufta Bakht Ahsan, Todd Pfleiger, Rishi Sinha, Anurag Gupta, Mihail Tarta, Mark Fussell, Vipul Modi, Mansoor Mohsin, Ray Kong, Anmol Ahuja, Oana Platon, Alex Wun, Matthew Snider, Chacko Daniel, Dan Mastrian, Yang Li, Aprameya Rao, Vaishnav Kidambi, Randy Wang, Abhishek Ram, Sumukh Shivaprakash, Rajeet Nair, Alan Warwick, Bharat S. Narasimman, Meng Lin, Jeffrey Chen, Abhay Balkrishna Mhatre, Preetha Subbarayalu, Mert Coskun, and Indranil Gupta. 2018. Service Fabric: A Distributed Platform for Building Microservices in the Cloud. In *Proceedings of the Thirteenth EuroSys Conference 2018* (Porto, Portugal) *(EuroSys '18)*. Association for Computing Machinery, New York, NY, USA, 1–15. https://doi.org/10.1145/3190508.3190546

[30] Max Karsten, Andrei Calin Dragomir, Radu Apsan, Vincenzo Stoico, and Ivano Malavolta. 2025. Experiment Runner: a Tool for the Automatic Orchestration of Experiments Targeting Software Systems. *Science of Computer Programming* (2025), 103415.

[31] Sven Köhler, Lukas Wenzel, Andreas Polze, Benedict Herzog, Henriette Hofmeier, Manuel Vögele, and Timo Hönig. 2023. Carbon-Aware Memory Placement. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems (HotCarbon '23)*. ACM, Boston, MA, USA. https://doi.org/10.1145/3604930.3605714

[32] Rutger Kool, Michiel Overeem, and Ivano Malavolta. 2025. Replication package of this study. https://github.com/S2-group/afas-sci-case-study-2025

[33] Ivano Malavolta, Vincenzo Stoico, and Patricia Lago. 2024. Ten Years of Teaching Empirical Software Engineering in the Context of Energy-Efficient Software. In *Handbook on Teaching Empirical Software Engineering*, Daniel Mendez, Davide Fucci, Marcos Kalinowski, Michael Felderer, Dietmar Pfahl, Rodrigo Spinola, Alessandra Bagnato, Valentina Lenarduzzi, and Davide Taibi (Eds.). Springer, 209–253.

[34] Henry B. Mann and Donald R. Whitney. 1947. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60. https://doi.org/10.1214/aoms/1177730491

[35] Hemant Kumar Mehta, Paul Harvey, Omer Rana, Rajkumar Buyya, and Blesson Varghese. 2020. WattsApp: Power-Aware Container Scheduling. In *Proceedings of the 13th IEEE/ACM International Conference on Utility and Cloud Computing (UCC)*. IEEE, 79–89. https://doi.org/10.1109/UCC48980.2020.00027

[36] Microsoft Corporation. 2024. *Playwright: Framework for Web Testing and Automation.* Microsoft Corporation. https://playwright.dev/ Accessed: 2025-05-10.

[37] Microsoft Corporation. 2025. Azure Service Fabric: A Distributed Systems Platform. https://azure.microsoft.com/en-us/products/service-fabric/. Accessed: 2025-06-28.

[38] Microsoft Corporation. 2025. Microsoft Azure: Cloud Computing Platform and Services. https://azure.microsoft.com/. Accessed: 2025-06-28.

[39] Microsoft Corporation. 2025. PowerShell: A Cross-platform Task Automation Solution. https://docs.microsoft.com/en-us/powershell/. Accessed: 2025-06-28.

[40] Microsoft Corporation. 2025. SQL Server: Relational Database Management System. https://www.microsoft.com/en-us/sql-server/. Accessed: 2025-06-28.

[41] Microsoft Corporation. 2025. Windows Server: The Platform for Building an Infrastructure of Connected Applications, Networks, and Web Services. https://www.microsoft.com/en-us/windows-server. Accessed: 2025-06-28.

[42] Adel Noureddine. 2022. PowerJoular and JoularJX: Multi-Platform Software Power Monitoring Tools. (June 2022), 1–4. https://doi.org/10.1109/IE54923.2022.9826760

[43] Priyavanshi Pathania, Rohit Mehra, Samarth Sikand, Vibhu Saujanya Sharma, Vikrant Kaulgud, Nikhil Bamby, Sanjay Podder, and Adam P. Burden. 2024. Towards a Framework for Carbon-aware Virtual Machine Management. In *Proceedings of the 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C '24)*. IEEE, 250–251. https://doi.org/10.1109/ICSA-C61636.2024.00052

[44] Karl Pearson. 1895. Note on Regression and Inheritance in the Case of Two Parents. *Proceedings of the Royal Society of London* 58, 347-352 (1895), 240–242.

[45] Filip Pizlo, Erez Petrank, and Bjarne Steensgaard. 2008. A Study of Concurrent Real-Time Garbage Collectors. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '08)*. ACM, Tucson, AZ, USA, 33–44.

[46] Proxmox Server Solutions GmbH. 2024. *Proxmox Virtual Environment: Open-source server virtualization platform*. Proxmox Server Solutions GmbH. https://www.proxmox.com/en/proxmox-virtual-environment Accessed: 2025-04-20.

[47] Giampaolo Rodola. 2024. *psutil: Cross-platform lib for process and system monitoring in Python*. https://github.com/giampaolo/psutil Accessed: 2025-05-12.

[48] Md. Faiyaz Abdullah Sayeedi, Jannatul Ferdous Deepti, Anas Mohammad Ishfaqul Muktadir Osmani, Taimur Rahman, Safrin Sanzida Islam, and Md. Motaharul Islam. 2024. A Comparative Analysis for Optimizing Machine Learning Model Deployment in IoT Devices. *Applied Sciences* 14, 13 (2024), 5459. https://doi.org/10.3390/app14135459

[49] Andreas Schmidt, Gregory Stock, Robin Ohs, Luis Gerhorst, Benedict Herzog, and Timo Hönig. 2024. carbond: An Operating-System Daemon for Carbon Awareness. *ACM SIGENERGY Energy Informatics Review* 4, 3 (2024), 52–57. https://doi.org/10.1145/3698355.3698364

[50] Victor Schmidt, Kamal Goyal, Aditya Joshi, Boris Feld, Liam Conell, Nikolas Laskaris, Doug Blank, Jonathan Wilson, Sorelle Friedler, and Sasha Luccioni. 2021. *CodeCarbon: Estimate and Track Carbon Emissions from Machine Learning Computing*. https://github.com/mlco2/codecarbon Open-source Python package for tracking carbon emissions of computational processes.

[51] Samuel Sanford Shapiro and Martin B. Wilk. 1965. An Analysis of Variance Test for Normality (Complete Samples). *Biometrika* 52, 3/4 (1965), 591–611. https://doi.org/10.1093/biomet/52.3-4.591

[52] Marina Shimchenko, Mihail Popov, and Tobias Wrigstad. 2022. Analysing and Predicting Energy Consumption of Garbage Collectors in OpenJDK. In *Proceedings of the 19th International Conference on Managed Programming Languages and Runtimes (MPLR '22)*. ACM, Brussels, Belgium, 3–15. https://doi.org/10.1145/3546918.3546925

[53] Oleh V. Talaver and Tetiana A. Vakaliuk. 2024. Dynamic System Analysis Using Telemetry. In *Proceedings of CS&SE@SW 2023 (CEUR Workshop Proceedings, Vol. 193)*. CEUR-WS.org. https://ceur-ws.org/Vol-193/

[54] WattTime. 2023. WattTime API: Real-Time Marginal Emissions Data. https://www.watttime.org/. Accessed: 2025-06-28.

[55] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering* (1st ed.). Springer.